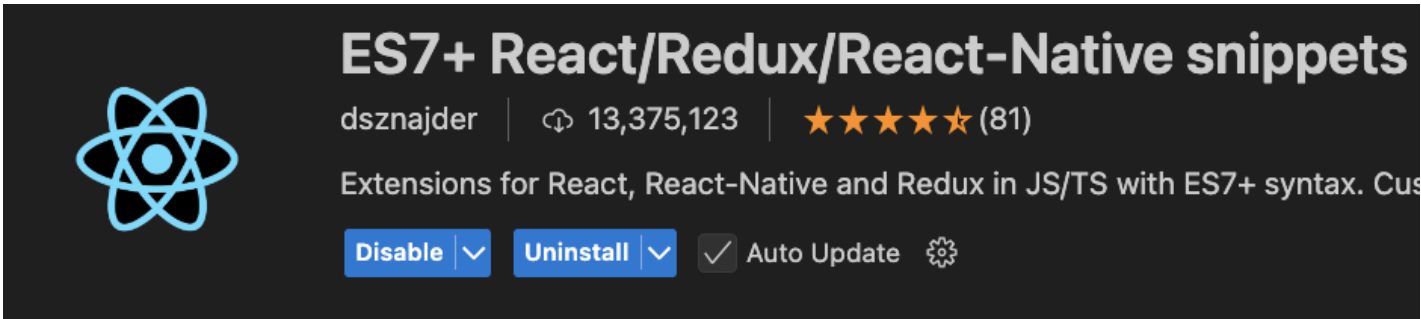


REACT JS 🥷

INSTALLATION DE L'APP ➡️ 😊

0 ➡️ 🥷 : Installez l'extension suivante dans Visual Studio Code :



0.1 ➡️ 🥷 : **Ensuite on va avoir besoin d'installer l'extension "React Developer Tools".**

🥷 : C'est une extension de navigateur qui nous permettra :



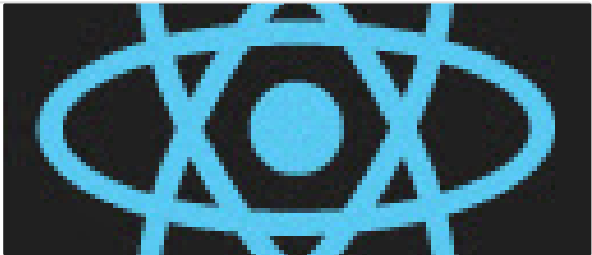
- D'inspecter et de visualiser les composants React pour faciliter leur débogage.
- Analyse des props et du state ➡️ permet d'examiner et de modifier les propriétés et l'état des composants en direct.
- Performance profiling ➡️ permet d'analyser et d'identifier les problèmes de performance dans votre application.
- Débogage ➡️ facilite la détection et la correction des erreurs dans votre application React pendant le développement.

🥷 : Installer React Developer Tools Chrome :

React Developer Tools - Chrome Web Store

Adds React debugging tools to the Chrome Developer Tools. Created from revision c7c68ef842 on 10/15/2024.

 <https://chromewebstore.google.com/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi?hl=en>

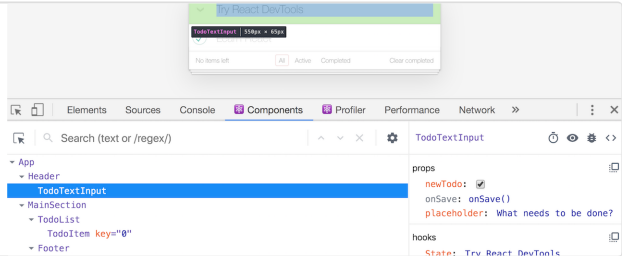


🥷 : Installer React Developer Tools Firefox :

React Developer Tools – Get this Extension for 🦊 Firefox (en-US)

Download React Developer Tools for Firefox. React Developer Tools is a tool that allows you to inspect a React tree, including the component hierarchy, props, state, and more. To get started, just open the Firefox devtools and switch to the "🔧 Components" or "🔧 Profiler" tab.

 <https://addons.mozilla.org/en-US/firefox/addon/react-devtools/>



🥷 : Si vous vous êtes perdu dans Edge :

React Developer Tools - Microsoft Edge Addons

Make Microsoft Edge your own with extensions that help you personalize the browser and be more productive.

 <https://microsoftedge.microsoft.com/addons/detail/react-developer-tools/gpphkfbcpiddadnolkpfckpihlkkil>

👤 : Et safari ?

👤 : Safari... désolé, mais les dev expérimentés préfèrent Chrome ou Firefox. Edge, c'est acceptable, mais pas idéal.

👤 : ok... 😊.

1 ⇒ 👤 : Positionnez-vous via votre terminal à la racine de votre dossier qui regroupe les chapitres "**00-API_Express**" & "**01-LA_FOIRE**", puis exécutez la commande suivante pour générer le projet React :

```
npx create-react-app front
```

2 ⇒ 👤 : Positionnez-vous dans l'application **React** en exécutant la commande suivante.

```
cd front
```

2.1 ⇒ 👤 : Installez la dépendance suivante (elle nous permettra de mettre en place un système de navigation) :

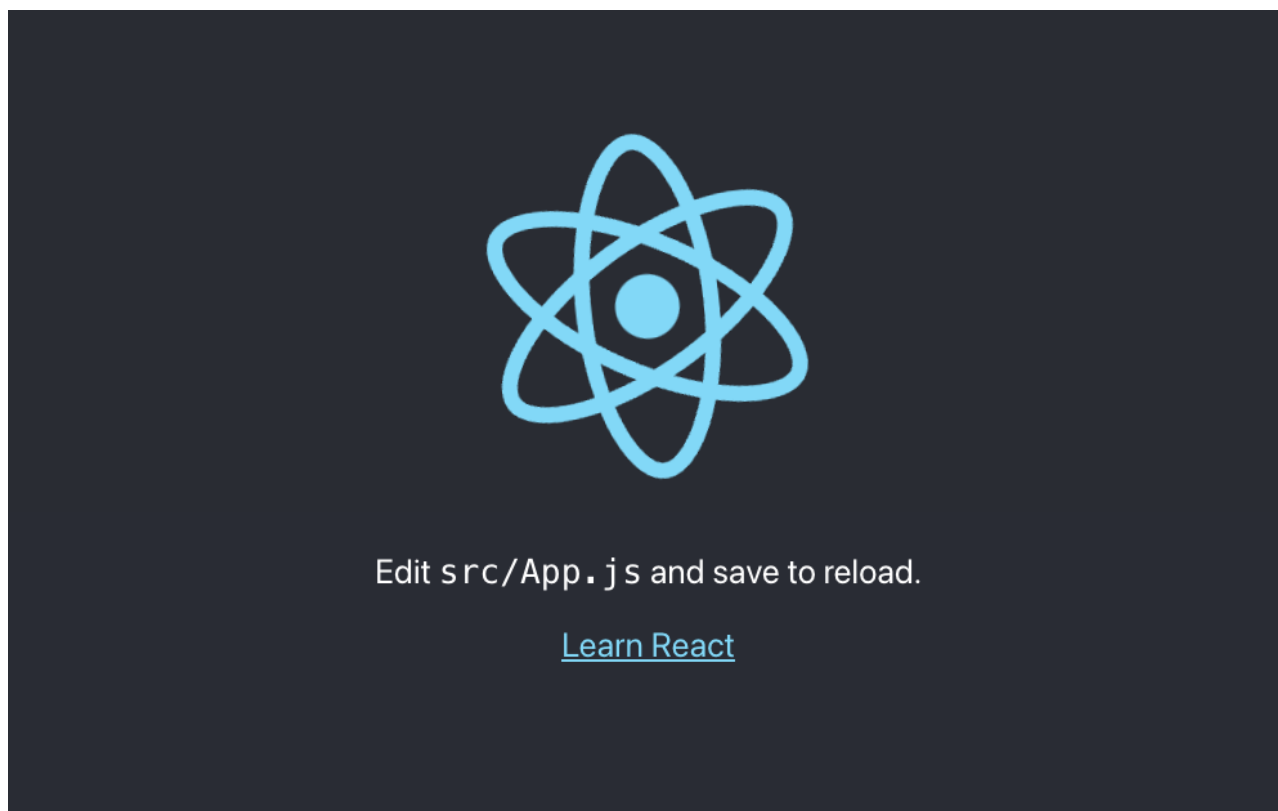
```
npm install react-router-dom
```

3 ⇒ 👤 : Une fois dans le dossier "**front**", démarrez le serveur **React** avec la commande suivante.

```
npm start
```

👤 : Votre serveur s'exécutera et tournera sur le port 3000 par défaut.

👤 : Une page s'ouvrira et vous affichera ceci :



👤 : Tout l'affichage à l'écran provient du fichier "**App.js**" situé dans le dossier "**src**".

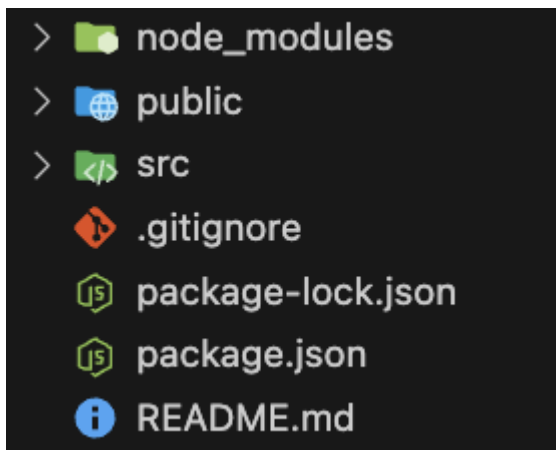
4 ⇒ 👤 : Arborescence de l'application

👤 : **Dossier public :**

- Contient les fichiers statiques accessibles publiquement
- Héberge le fichier index.html principal
- Stocke les ressources comme les images, les polices ou les fichiers robots.txt

👤 : **Dossier src :**

- Contient tout le code source de l'application
- Inclut les composants React, les fichiers JavaScript/JSX
- Stocke les styles (CSS), les tests et autres fichiers de logique
- C'est ici que se passe la majorité du développement



5 ⇒ 👤 : Dans le dossier "src", créez deux sous-dossiers "**components**" et "**pages**".

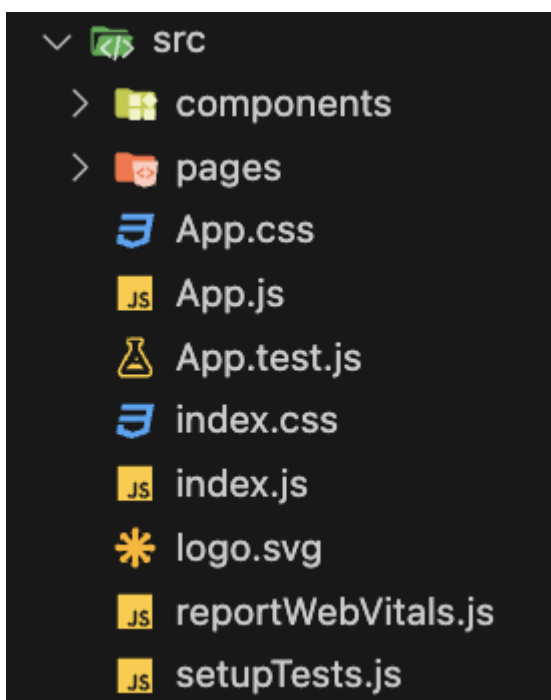
👤 : Le Dossier Components

💡 Contiendra tous les composants réutilisables.

👤 : Le Dossier Page

💡 Contiendra toutes les pages de votre application.

👤 : **Resultat attendu :**



DEMARRAGE ⇒ 🚀 😊

6 ⇒ 👤 : Dans le dossier "**pages**", créez un fichier "**home.jsx**".

Puis mettez en place le code suivant sans les commentaires.

```
1  // Déclaration du composant Home en tant que fonction (HOOK)
2  // Un composant React est une fonction ou une classe qui
   retourne du JSX (un mélange de JavaScript et de HTML)
3  const Home = () => {
4    // Le JSX retourné par le composant sera affiché à l'écran
5    // Il doit être contenu dans une seule balise parent, comme un
   fragment React (<> </>) ou une div, etc.
6
7    return (
8      <>
9        {/* Ici, vous pouvez ajouter le contenu HTML ou d'autres
10         composants que vous souhaitez afficher */}
11        {/* Par exemple : */}
12        {/* <h1>Bienvenue sur ma page d'accueil</h1> */}
13      </>
14    );
15
16   // Exporte le composant pour pouvoir l'utiliser dans d'autres
   fichiers
17   export default Home;
```

6.1 ⇒ 👤 : Décommentez la ligne 11 comme ceci

```
8      <>
9        {/* Ici, vous pouvez ajouter le contenu HTML ou d'autres
10         composants que vous souhaitez afficher */}
11        {/* Par exemple : */}
12        <h1>Bienvenue sur ma page d'accueil</h1>
13      </>
```

PACKAGE ROUTEUR ⇒ 🏎️ 😊

7 ⇒ 👤 : Faisons en sorte d'afficher le contenu de notre **composant**, mais pour cela nous avons besoin de configurer le système de "**route**".

👤 : Modifions le fichier "**index.js**" avec le code suivant :

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
```

```

import reportWebVitals from './reportWebVitals';
// Importe BrowserRouter depuis la bibliothèque 'react-router-dom'
// BrowserRouter est un composant
// qui permet de gérer la navigation (routing) dans une application React
import { BrowserRouter } from 'react-router-dom';

// Trouve l'élément HTML avec l'ID 'root' dans le DOM
// Cela correspond généralement à une div
// dans le fichier index.html (dans le dossier public )
// où l'application React sera injectée
const root = ReactDOM.createRoot(document.getElementById('root'));

// Utilise la méthode render() pour afficher l'application dans le DOM
root.render(
  // React.StrictMode est un outil de développement
  // qui permet de détecter des problèmes potentiels dans le code
  // Il n'a aucun effet sur la production,
  // mais aide à écrire un code React plus propre
  <React.StrictMode>
    {/* BrowserRouter encapsule l'application
    pour activer la gestion des routes */}
    <BrowserRouter>
      {/* App est le composant principal de l'application */}
      {/* C'est ici que sont définies les routes
      et le contenu de l'application */}
      <App />
    </BrowserRouter>
  </React.StrictMode>
);

reportWebVitals();

```

8 ⇒  : Le résultat de votre fichier "App.js" doit être identique à l'image ci-dessous.

```

1  // Importe les composants Routes et Route depuis 'react-router-dom'
2  // - Routes agit comme un conteneur pour définir les différentes routes de l'application
3  // - Route définit une route spécifique et le composant qui sera affiché pour cette route
4  import { Routes, Route } from 'react-router-dom';
5
6  // Importe le fichier CSS global pour styliser l'application
7  import './App.css';
8
9  // Importe le composant Home depuis le fichier 'pages/home'
10 // Ce composant sera affiché pour la route principale
11 import Home from './pages/home';
12
13 function App() {
14   // Le composant App contient les définitions
15   // des routes de l'application
16   return (
17     <Routes>
18       {/* Définition d'une route */}
19       {/* index signifie que cette route
20        correspond au chemin de base ('/') */}
21       {/* element spécifie le composant
22        à afficher pour cette route */}
23       <Route index element={<Home />} />
24     </Routes>
25   );
26 }
27
28 export default App;

```

👤 : En vérifiant votre page maintenant, vous devriez obtenir ce résultat.

Bienvenue sur ma page d'accueil

PAGE HOME ⇒ 🏠 😊

9 ⇒ 👤 : Mettez à jour le fichier "home.jsx" avec les modifications ci-dessous.

```
1 // Importe les hooks `useState` et `useEffect` depuis React
2 // - `useState` permet de gérer l'état dans un composant fonctionnel
3 // - `useEffect` permet d'effectuer des effets secondaires (par exemple :
  appels API, mises à jour après le rendu, etc.)
4 import { useState, useEffect } from "react";
5
6 const Home = () => {
7   // Déclare un état pour stocker une liste d'articles
8   // - `articles` : la variable qui contient la liste
9   // - `setArticle` : la fonction utilisée pour mettre à jour la liste
10  // Initialement, `articles` est un tableau vide
11  const [articles, setArticle] = useState([]);
12
13  // Déclare un état pour gérer les erreurs
14  // - `error` : la variable qui contient le message d'erreur
15  // - `setError` : la fonction utilisée pour mettre à jour l'erreur
16  // Initialement, `error` est `null` (pas d'erreur)
17  const [error, setError] = useState(null);
18
19  return (
20    <>
21      <h1>Bienvenue sur ma page d'accueil</h1>
22    </>
23  );
24 };
25
26 export default Home;
```

10 ⇒ 👤 : Mettre en place le `useEffect` pour appeler notre API


```

1  import { useState, useEffect } from "react";
2
3  const Home = () => {
4    const [articles, setArticle] = useState([]);
5    const [error, setError] = useState(null);
6
7    // Le hook useEffect est utilisé pour effectuer des actions secondaires, comme des appels API ou des mises
    // à jour après le premier rendu du composant
8    // L'effet s'exécutera lorsque le composant sera monté (au chargement de la page)
9    useEffect(() => {
10     // Fonction asynchrone fetchArticle qui récupère les articles depuis l'API
11     const fetchArticle = async () => {
12       try {
13         // On effectue un appel HTTP GET pour récupérer les articles à partir de l'API à l'adresse 'http://
        localhost:8000/api/article/all'
14         const response = await fetch(`http://localhost:8000/api/article/all`);
15
16         // Si la requête est réussie, les données de l'API sont converties en JSON
17         const data = await response.json();
18
19         // Les articles récupérés sont stockés dans l'état 'articles' en utilisant "setArticle"
20         // Cela déclenche un nouveau rendu du composant (refresh de la page) avec les articles récupérés
21         setArticle(data);
22       } catch (error) {
23         // Si une erreur se produit (par exemple, un problème de réseau ou de réponse), elle est capturée
        // dans le bloc catch
24         // L'erreur est stockée dans l'état 'error' pour pouvoir l'afficher à l'utilisateur
25         setError(error.message);
26       }
27     };
28
29     // Appel de la fonction fetchArticle
30     fetchArticle();
31   }, []);
32
33   if(error) return <> <p>{error}</p> </>
34
35   return (

```

```

37   <>
38     <h1>Bienvenue sur ma page d'accueil</h1>
39   </>
40 );
41 };
42
43 export default Home;

```

11 ⇒ 🧑 : Vérifier que vous récupérez bien les articles provenant de votre API

🧑 : Comment peut-on vérifier les données ?

🧑 : Ajoute un console.log() juste après la ligne 12.

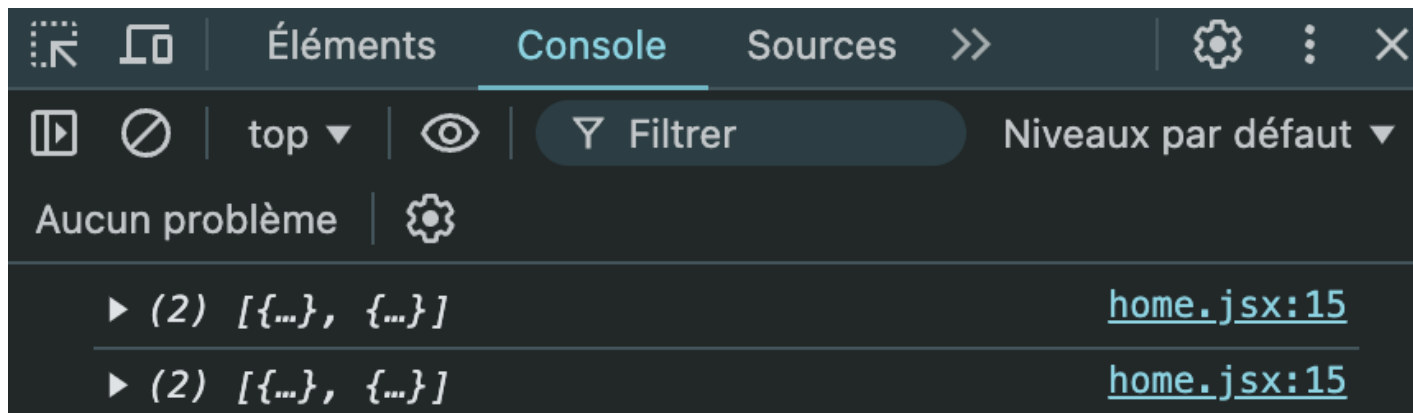
🧑 : Exact comme ceci :

```

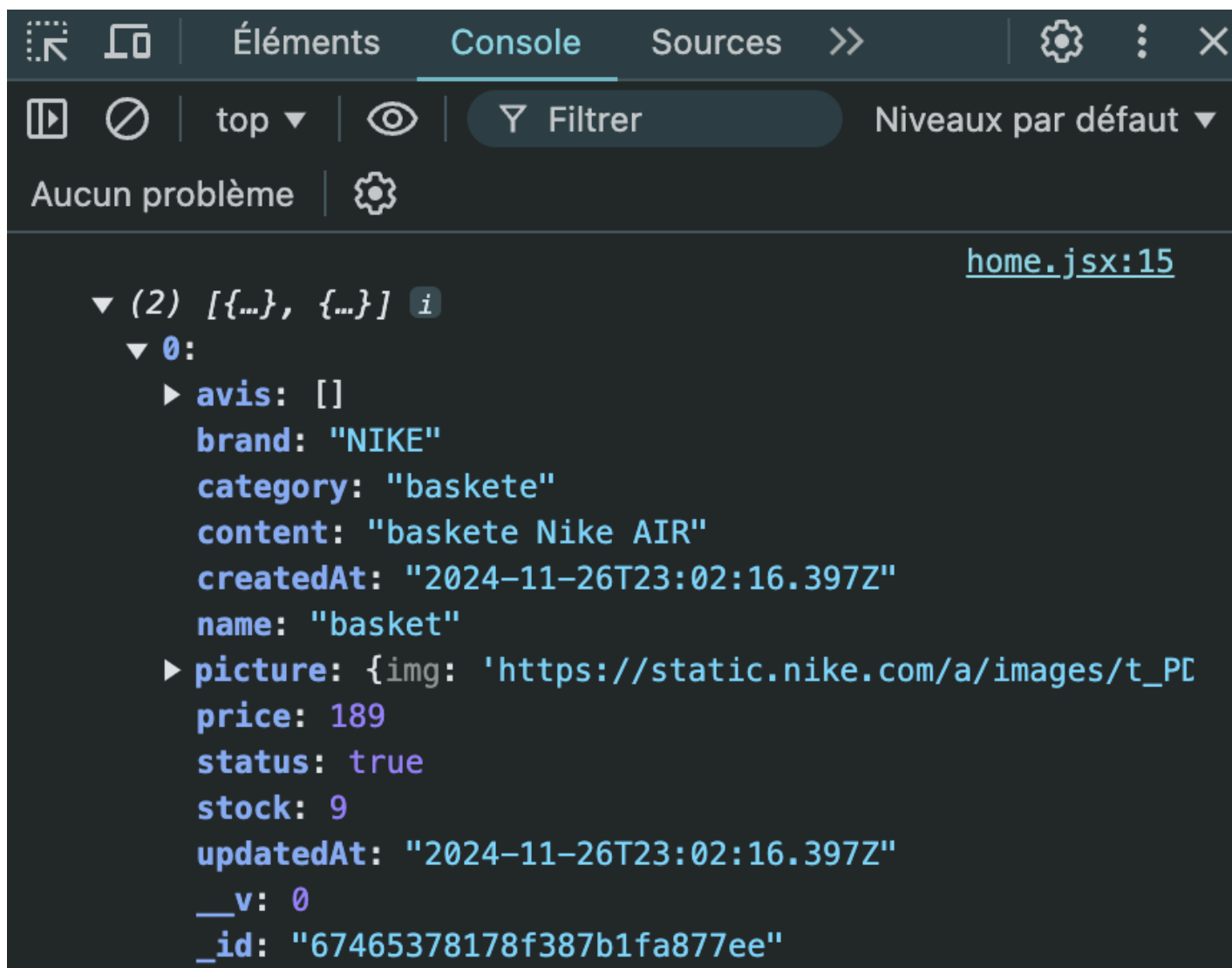
7    useEffect(() => {
8
9      const fetchArticle = async () => {
10        try {
11          const response = await fetch(`http://localhost:8000/api/article/all`);
12          const data = await response.json()
13
14          // LOG POUR VERIFIER LES DONNEES RECUPERER DE NOTRE API
15          console.log(data);
16
17          setArticle(data);
18        } catch (error) {
19          setError(error.message);
20        }
21      };
22      fetchArticle();
23    }, []);

```

11.1 ⇒ 🧑 : Vérifiez votre inspecteur, vous devriez avoir quelque chose qui ressemble à ceci :



👤 : Vous pouvez déplier les objets :



👤 : Maintenant que nous avons réussi à récupérer les données, quelle serait la prochaine étape logique ?

👤 : Une boucle pour les afficher ?

👤 : Non !

👤 : 🤔 ...

👤 : J'plaisante c'est bien ça Mimiche ! on va mettre en place une boucle "map" pour afficher nos articles.


```
42 <>
43 <h1>Bienvenue sur ma page d'accueil</h1>
44 {articles.map((item) => (
45   // La méthode map() parcourt chaque élément de la liste `articles` et retourne un
46   // 'item' représente un article
47
48   <div key={item._id}>
49     {/*
50      'key' est une propriété spéciale utilisée par React pour identifier de manière
51      unique chaque élément dans une liste */}
52     {/* Cela permet à React de mieux gérer la mise à jour de la liste et d'optimiser le rendu */}
53
54     <p>{item.name}</p>
55     {/* Affiche le nom de l'article en utilisant la propriété 'name' de chaque 'item' */}
56
57     <img src={item.picture.img} width={200} />
58     {/* Affiche l'image de l'article en utilisant la propriété 'picture.img' de chaque 'item' */}
59
60     <p>{item.price}</p>
61     {/* Affiche le prix de l'article en utilisant la propriété 'price' de chaque 'item' */}
62   </div>
63   )})
64 </>
```

👤 : Vous devriez avoir le même résultat que moi sur votre page :

Bienvenue sur ma page d'accueil

basket



189

Femme



149

👤 : Mes images ne s'affiche pas !

👤 : C'est parce que les URLs ne sont pas bonne, modifiez les via postman en t'assurant de bien récupérer l'adresse d'une image en ligne.

PAGE DETAIL ⇒ 🕵️🤔

12 ⇒ 🕵️ Bravo 🎉 vous arrivez à communiquer avec votre API que vous avez vous-même créée tel un développeur **fullstack**, mais ce n'est pas fini...it... , on va maintenant faire en sorte de pouvoir cliquer sur un article, afin d'afficher plus d'informations sur l'article sélectionné !

🕵️ : Dans le dossier "**pages**" créez un fichier "**detail.jsx**"

```
02-FRONT > src > pages > detail.jsx > default
1
2  const Detail = () => {
3
4    return(
5      <>
6        <h1>Page Detail</h1>
7      </>
8    );
9  };
10
11  export default Detail;
```

🕵️ : Il faut lui créer une route maintenant ?

🕵️ : yes ! dans le fichier "**App.js**" on va importer le composant et l'ajouter dans le router comme ceci :

```
1
2  import { Routes, Route } from 'react-router-dom';
3
4  import './App.css';
5
6
7  import Home from './pages/home';
8
9  // Importe le composant Detail depuis le fichier 'pages/detail'
10 import Detail from './pages/detail';
11
12
13 function App() {
14   return (
15     <Routes>
16       <Route index element={<Home />} />
17       /*
18        Définit une route dynamique avec un paramètre 'id' dans l'URL
19        `path='/detail/:id` : Cette route correspond à l'URL '/detail/{id}',
20        {id} qui s'adapte en fonction de l'article qu'on sélectionnera.
21        `:id` est un paramètre de l'URL que nous pouvons utiliser dans le composant
22        pour récupérer et afficher des données spécifiques
23        (par exemple, l'article avec l'ID donné)
24       */
25       <Route path='/detail/:id' element={<Detail/>} />
26     </Routes>
27   );
28 }
29
30 export default App;
```

13 ⇒ 👤 Maintenant que la route dynamique a été créée, nous allons pouvoir sélectionner un article pour rediriger vers la page "detail.jsx"

👤 : Ça me dit quelque chose...

👤 : On la fait en ajax avec JQuery...

👤 : C'est ça ! Du coup, on va envelopper la balise "img" avec un lien de redirection, et dans ce lien on spécifiera l'ID de l'article pour l'envoyer dans l'URL.

👤 : Comme on faisait avec Postman ?

👤 : Oui ce sera la même principe, regardez...

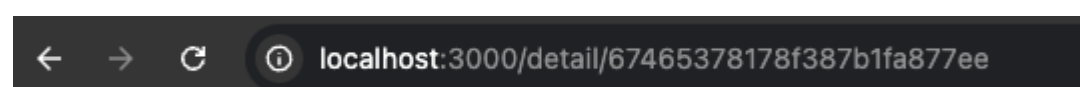
👤 : Dans le fichier "home.jsx", importez le composant "Link" depuis "react-router-dom"

```
import { Link } from "react-router-dom";
```

👤 : Maintenant, utilisons ce composant pour envelopper la balise "img" située dans notre "map".

```
44 {articles.map((item) => (  
45   <div key={item._id}>  
46     <p>{item.name}</p>  
47     /*  
48     Le composant Link est utilisé pour naviguer entre les pages  
49     de l'application sans recharger la page complète  
50     Il remplace les liens classiques (<a>) pour une navigation React  
51     */  
52     <Link to={{ pathname: `/detail/${item._id}` }}>  
53       /* L'attribut 'to' définit l'URL vers laquelle le lien va rediriger */  
54       /* Ici, l'URL dynamique est construite en utilisant 'item._id' */  
55       /* 'item._id' représente l'id de l'article, qui sera retranscrit dans l'URL */  
56       <img src={item.picture.img} width={200} />  
57     </Link>  
58     <p>{item.price}</p>  
59   </div>  
60 )})
```

👤 : Maintenant vous pouvez cliquer sur l'image de l'un de vos articles et ceci devrait s'afficher dans votre URL :



👤 : Puis sur votre écran :

Page Detail

14 ⇒ 🧑 : Dans la page "details.jsx" On va maintenant avoir besoin d'importer les hooks "useState & useEffect"

```
import { useEffect, useState } from "react";
```

🧑 : On peut maintenant créer un state qui contiendra l'article ?

🧑 : En effet, comme ceci :

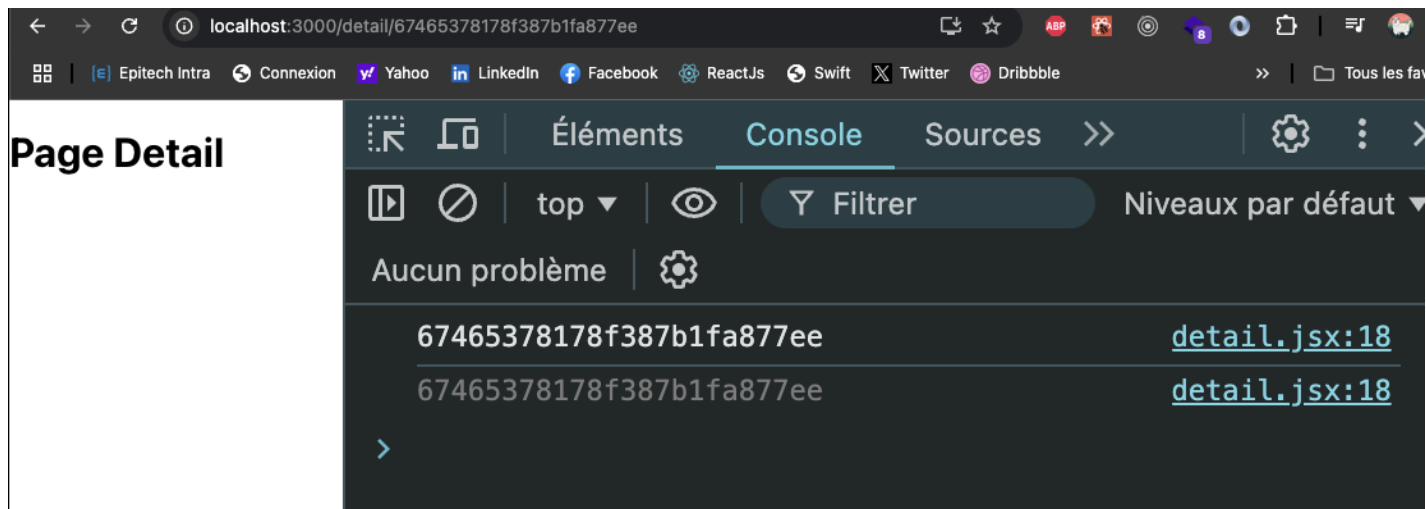
```
1  import { useEffect, useState } from "react";
2
3  const Detail = () => {
4
5      const [article, setArticle] = useState([])
6      const [error, setError] = useState(null);
7
8      return(
9          <>
10         | <h1>Page Detail</h1>
11         </>
12     );
13 };
14
15 export default Detail;
```

🧑 : On peut faire le "useEffect" maintenant ?

🧑 : Avant d'effectuer le "useEffect", on doit d'abord récupérer l'ID que nous avons envoyé dans l'URL comme ceci :

```
1  import { useEffect, useState } from "react";
2
3  // Importation des hooks `useParams` et `useLocation` de 'react-router-dom'
4  // - `useParams` permet d'extraire les paramètres dynamiques de l'URL (comme l'id)
5  import { useParams } from 'react-router-dom';
6
7  const Detail = () => {
8
9      const [article, setArticle] = useState([])
10     const [error, setError] = useState(null);
11
12     // `useParams` permet d'accéder aux paramètres dynamiques de l'URL
13     // il retourne un objet qui contient tous les paramètres de l'URL définis dans la route.
14     // Extraction du paramètre 'id' de l'objet `params`
15     // Dans l'URL '/detail/:id', 'id' correspond au paramètre dynamique
16     const { id } = useParams();
17
18     return(
19         <>
20         | <h1>Page Detail</h1>
21         </>
22     );
23 };
24
25 export default Detail;
```

🧑 : Faites un console.log() de l'id après l'extraction de l'id et vérifiez le résultat, vous devriez avoir ça :



👤 : On peut maintenant appeler notre API en spécifiant l'id de l'article que l'on souhaite récupérer ?

👤 : yes ! À la suite du "useParams" on va ajouter le "useEffect" comme ceci :

```

11  useEffect(() => {
12    // On récup les données de l'API pour un article spécifique
13    const fetchArticle = async () => {
14      try {
15        // On effectue une requête GET à l'API pour récupérer l'article avec l'ID spécifié dans l'URL
16        const response = await fetch(`http://localhost:8000/api/article/${id}`);
17
18        // Si la réponse est valide, la réponse est convertie en JSON
19        const data = await response.json();
20
21        // Met à jour l'état 'article' avec les données récupérées
22        setArticle(data);
23      } catch (error) {
24        // Si une erreur se produit, met à jour l'état 'error' avec le message d'erreur
25        setError(error.message);
26      }
27    };
28
29    fetchArticle(); // Appelle la fonction fetchArticle pour récupérer les données
30
31  }, [id]); // Le tableau de dépendances [id] signifie que l'effet se réexécute à chaque fois que l'ID change (ex:
           lorsqu'un utilisateur navigue vers un autre article)

```

15 ⇒ 👤 : On peut faire la map maintenant !

👤 : Non ! on a qu'un seul article on n'a pas besoin de faire de boucle.

👤 En effet ! On peut directement récupérer les données, je vais juste m'assurer que picture ait terminé de charger ses images avant de les afficher comme ceci :

```

// lorsqu'un utilisateur navigue vers un autre article

if (error) {
  return <p>Error: {error}</p>;
}

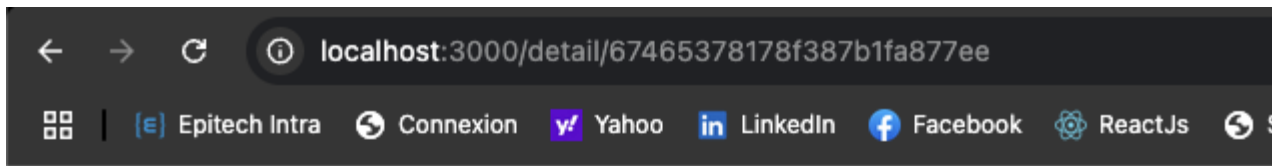
return(
  <>
    <h1>Détails de l'article</h1>
    <h2>{article.name}</h2>
    <img src={article.picture?.img} alt={article.name} width={200} />
    {/*
      - La syntaxe `article.picture?.img` utilise l'opérateur de chaînage optionnel (`?.`)
      pour éviter une erreur si `article.picture` est `undefined` ou `null`.
      - Si `article.picture` n'existe pas ou est `null`,
      l'image ne sera pas affichée, ce qui permet de gérer les erreurs de données manquantes.
    */}
    <p>{article.price}€</p>
    <p>{article.description}</p>
  </>
);

```

👤 : Ça ne fonctionne pas, j'ai une erreur 404 ! dans mon inspecteur.

👤 : Si vous avez une erreur **404**, c'est que l'URL que vous appelez n'est pas bonne, donc vérifiez-la.

👤 : Oui maintenant ça fonctionne, j'ai dû modifier l'url en `api/article/get/${id}{id}``



Détails de l'article

basket



189€

PAGE ADD ⇒ 🤔

16 ⇒ 👤 : Mettre en place une page « add » (pour pouvoir ajouter un ou des articles.

👤 : Dans le dossier "**pages**" créons une page "**add.js**" puis mettons en place la structure par défaut de ce composant comme ceci :

```
import React, { useState } from 'react';

const AddArticle = () => {

  return (
    <>
      <h1>ADD ARTICLE</h1>
    </>
  );
};

export default AddArticle;
```

👤 : **Pensez à créer sa route et ajouter le lien de navigation dans votre header pour pouvoir accéder à cette page.**

👤 : Maintenant que les routes sont établies, mettons en place un state "article" qui nous permettra de stocker les informations. Et un tableau contenant le nombre d'images associées à nos article.

```
const imgInput = ['img', 'img1', 'img2', 'img3', 'img4']
const [article, setArticle] = useState({
  name: '',
  content: '',
  category: '',
  brand: '',
  price: 0,
  picture: {
    img: '',
    img1: '',
    img2: '',
    img3: '',
    img4: ''
  },
  status: true,
  stock: 0
});
```

👤 : Mettons en place la fonction "handleChange" pour mettre à jour notre state "article"

```
const handleChange = (e) => {
  const { name, value } = e.target;
  if (name.startsWith('img')) {
    // prev est un paramètre de la fonction de callback
    // passée à setArticle qui représente
    // la valeur précédente (actuelle) de l'état (state)
    // avant la mise à jour
    // assurer que nous mettons à jour l'état en nous
    // basant sur sa version la plus récente
    setArticle(prev => ({ ...prev, picture: {
      ...prev.picture,
      [name]: value
    } }));
  } else {
    setArticle(prev => ({ ...prev, [name]: value }));
  }
};
```



👤 : `startsWith()` est une méthode JavaScript qui vérifie si une chaîne de caractères commence par une sous-chaîne spécifique. On l'utilise pour vérifier si le nom du champ (`name`) commence par "img". Dans notre contexte spécifique, on vérifie si le champ en cours de modification est lié à une image. Si oui, on met à jour la propriété `picture` de l'objet article.

👤 : Mettons maintenant en place la méthode "handleSubmit" qui sera appelée après la validation du formulaire.


```
const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    const response = await axios.post(`http://localhost:8000/api/article/add`, article)
    console.log(response);
  } catch (error) {
    console.error(error.message);
  }
};
```

👤 : Verifier bien les urls de votre "API", si vous aviez une erreur 404 réadapter l'url de l'image précédente. Puis mettons en place le formulaire dans notre composant "add.jsx".

```
<form onSubmit={handleSubmit}>
  <input
    type="text"
    name="name"
    onChange={handleChange}
    placeholder="Nom de l'article"
    required
  />
  <textarea
    name="content"
    onChange={handleChange}
    placeholder="Description"
    required
  />
  <input
    type="text"
    name="category"
    onChange={handleChange}
    placeholder="Catégorie"
    required
  />
  <input
    type="text"
    name="brand"
    onChange={handleChange}
    placeholder="Marque"
    required
  />
  <input
    type="number"
    name="price"
    onChange={handleChange}
    placeholder="Prix"
    required
  />

  {imgInput.map((imgName, index) => (
    <div key={imgName}>
      <label>
        {index === 0 ?
          'Image principale (URL):'
          :
          `Image ${index} (URL):`}
      </label>
    </div>
  )
  )}
```

```

        </label>
        <input
          type="text"
          name={imgName}
          onChange={handleChange}
          // slice prend le dernier caractère du nom de l'image
          // (par exemple, pour 'img1', il extrait '1', ok ok ;) ? )
          placeholder={`Image ${imgName.slice(-1)} `}
        />
      </div>
    )))
    <input
      type="number"
      name="stock"
      onChange={handleChange}
      placeholder="Stock"
      required
    />
    <input
      type="checkbox"
      name="status"
      checked={article.status}
      onChange={e => setArticle(prev => (
        {...prev, status: e.target.checked}
      ))}
    />

    <button>Ajouter l'article</button>
  </form>

```



👤 : Le checkbox est utilisé pour gérer le statut de l'article.

- Il utilise la propriété "checked" qui est liée à article.status
- Lors du changement d'état (onChange), il utilise une fonction qui met à jour le state "article" en modifiant uniquement la propriété "status" avec la nouvelle valeur cochée/décochée (e.target.checked)

👤 : La syntaxe utilisée est une mise à jour du state avec le spread operator (...prev) qui conserve toutes les autres propriétés de l'article tout en modifiant uniquement le status.

VERSION AVEC UPLOAD ⇒ 🥰

Pour mettre en place un formulaire dynamique, nous allons initialiser un tableau. Si vous l'avez déjà créé, vous pouvez passer cette étape.

```
const imgInput = ["img", "img1", "img2", "img3", "img4"];
```

Modifiez la structure de votre state 'article' selon l'exemple ci-dessous.

```
const [article, setArticle] = useState({
  name: "",
```

```

    content: "",
    category: "",
    brand: "",
    price: 0,
    img: [],
    status: true,
    stock: 0,
  });

```

Adaptez votre fonction '[handleSubmit](#)' avec le code ci-dessous. version pour '[Axios](#)'.

```

// Fonction asynchrone pour gérer la soumission du formulaire
const handleSubmit = async (e) => {
  // Empêche le comportement par défaut du formulaire
  e.preventDefault();

  // Création d'un objet FormData pour envoyer
  // des données multipart/form-data
  // Permet l'envoi de fichiers et de données textuelles
  const formData = new FormData();

  // Ajout des champs du formulaire dans FormData
  formData.append("name", article.name);
  formData.append("content", article.content);
  formData.append("category", article.category);
  formData.append("brand", article.brand);
  // parseInt permet de faire une conversion
  // des valeurs numériques en entiers
  formData.append("price", parseInt(article.price));
  formData.append("status", article.status);
  formData.append("stock", parseInt(article.stock));

  // Parcours du tableau d'images
  // et ajout de chaque image dans FormData
  // Le nom 'img' doit correspondre au champ
  // attendu par Multer côté serveur
  article.img.forEach((image) => {
    formData.append("img", image);
  });

  try {
    // Envoi de la requête POST avec axios
    // Headers spécifiques pour l'envoi de fichiers
    const response = await axios.post(
      `http://localhost:8000/api/article/add`,
      formData,
      {
        headers: {
          "Content-Type": "multipart/form-data",
        },
      }
    );
  } catch (error) {
    console.error(error.message);
  }
};

```

Version pour 'fetch'.

```
// Fonction asynchrone pour gérer la soumission du formulaire
const handleSubmit = async (e) => {
  // Empêche le comportement par défaut du formulaire
  e.preventDefault();

  // Création d'un objet FormData pour
  // envoyer des données multipart/form-data
  // Permet l'envoi de fichiers et de données textuelles
  const formData = new FormData();

  // Ajout des champs du formulaire dans FormData
  formData.append("name", article.name);
  formData.append("content", article.content);
  formData.append("category", article.category);
  formData.append("brand", article.brand);
  // parseInt permet de faire une conversion
  // des valeurs numériques en entiers
  formData.append("price", parseInt(article.price));
  formData.append("status", article.status);
  formData.append("stock", parseInt(article.stock));

  // Parcours du tableau d'images
  // et ajout de chaque image dans FormData
  // Le nom 'img' doit correspondre
  // au champ attendu par Multer côté serveur
  article.img.forEach((image) => {
    formData.append("img", image);
  });

  try {
    // Envoi de la requête POST avec fetch
    const response = await fetch('http://localhost:8000/api/article/add', {
      method: 'POST',
      body: formData
    });
    // Pas besoin de spécifier le Content-Type avec fetch
    // lors de l'utilisation de FormData
    // Il sera automatiquement défini avec la bonne valeur
    // incluant le boundary
  });

  if (!response.ok) throw new Error(`HTTP error! status: ${response.status}`);
} catch (error) {
  console.error('Error:', error);
}
};
```

Modifiez maintenant la fonction 'handleChange'.

```
// Fonction pour gérer les changements dans le formulaire
const handleChange = (e) => {
  // Destructuration pour extraire name, value et files de l'événement
  const { name, value, files } = e.target;
```

```

// Vérifie si le champ modifié est une image (commence par "img")
if (name.startsWith("img")) {
  // Mise à jour du state pour les images
  setArticle((prev) => ({
    ...prev, // Garde toutes les propriétés précédentes
    // Si des fichiers sont sélectionnés,
    // ajoute le premier fichier au tableau d'images
    // Sinon, on conserve le tableau d'images existant
    img: files ? [...prev.img, files[0]] : prev.img,
  }));
} else {
  // Pour les autres champs (pas les images)
  setArticle((prev) => ({
    // Garde toutes les propriétés précédentes
    ...prev,
    // Met à jour la propriété correspondante avec la nouvelle valeur
    [name]: value
  }));
}
};

```

Et maintenant il faut ajouter l'**input** qui va nous permettre de sélectionner une **image** dans le formulaire.

```

{imgInput.map((imgName, index) => (
  <div key={imgName}>
    <label>
      {index === 0 ? "Image principale (URL):" : `Image ${index} (URL):`}
    </label>
    <input
      type="file"
      name={imgName}
      onChange={handleChange}
      // slice prend le dernier caractère du nom de l'image
      // (par exemple, pour 'img1', il extrait '1')
      placeholder={`Image ${imgName.slice(-1)}`}
    />
  </div>
))}

```

 : Maintenant si vous ajoutez un article il devrait apparaître dans votre page de navigation.

PAGE UPDATE ⇒ 🤪

17 ⇒ Fonction permettant de **"supprimer"** un article à partir de la **"page"** **"detail"**.

 : Dans la page **"detail"**, **importer** les **useNavigate** et **Link**, s'ils sont manquants :

```
import { useParams, useNavigate, Link } from "react-router-dom";
```

 : Créons une constante **navigate** grâce à **"l'import"** de notre hook **"useNavigate"**.

Cette **constante** nous permettra de faire une **redirection** vers la page **"home"** après la suppression.

```

10   const { id } = useParams();
11   const navigate = useNavigate()
12

```

👤 : Ajouter une fonction 'deleteArticle' pour supprimer un article dans la page "detail".

```

const deleteArticle = async () => {
  try {
    const response = await axios.delete(`http://localhost:8000/api/article/
delete/${article._id}`);
    if (response.status === 200) navigate('/');
  } catch (error) {
    console.error('Erreur lors de la suppression:', error);
  }
}

```

👤 : Maintenant mettons en place un "bouton" qui nous permettra d'appeler notre fonction "deleteArticle" à la suite de l'affichage nos articles :

```
<button onClick={deleteArticle}>Supprimer l'article</button>
```

👤 : Maintenant, si vous cliquez sur le bouton, l'article sélectionné sera supprimé et vous serez redirigé vers la page "home"

18 ⇒ Mettre en place une page "update" pour modifier le contenu d'un article. Créez la structure par défaut du composant puis ajoutez sa route dans "app.js".



Ne pas ajouter de lien dans la navbar car cette page ne doit être accessible qu'à partir d'une sélection d'article.

👤 : Avant d'aller plus loin, ajoutons un lien de redirection à partir de la page "detail" qui nous permettra d'accéder à la page "update".



Cette étape est nécessaire car nous devons identifier un article spécifique avant de pouvoir le modifier.

👤 : Donc, à la suite du bouton delete, ajoutons cette ligne :

```
<Link to={{ pathname: `/update/${article._id}` }}>Update</Link>
```



👤 : Assurez que le lien fonction en cliquant dessus.

👤 : Dans la page 'update' ajoutons les import

```
1 import { useEffect, useState } from "react";
2 import { useParams, useNavigate } from "react-router-dom";
3 import axios from 'axios';
```

👤 : Récupérons l'ID de l'article sélectionné depuis l'URL du navigateur grâce au hook useParams, et créons une constante pour gérer la redirection après la modification de l'article.

```
7 const { id } = useParams();
8 const navigate = useNavigate();
```

👤 : Mettons en place un tableau qui définira le nombre d'images pouvant être stockées pour un article.

```
const imgInputs = ['img', 'img1', 'img2', 'img3', 'img4'];
```

👤 : Dans le composant ajoutons à la suite un state "article" qui sera similaire a celui de la page "add.js"

```
const [article, setArticle] = useState({
  name: '',
  content: '',
  category: '',
  brand: '',
  price: '',
  picture: {
    img: '',
    img1: '',
    img2: '',
    img3: '',
    img4: ''
  },
  status: false,
  stock: 0
});
```

👤 : Maintenant on va devoir appeler notre API pour récupérer les données de l'article sélectionné avec un "useEffect". C'est le même principe que pour la page "detail"

```
useEffect(() => {
  const fetchArticle = async () => {
    try {
      const response = await axios.get(`http://localhost:8000/api/article/get/${id}`);
      setArticle(response.data);
    } catch (error) {
      console.error(error.message);
    }
  };
  fetchArticle();
}, [id]);
```

👤 : Nous devons mettre à jour notre state "article" en utilisant la fonction "handleChange".


```
// Fonction qui gère les changements dans le formulaire
const handleChange = (e) => {
  // Destructure le nom et la valeur du champ modifié
  const { name, value } = e.target;

  // Vérifie si le champ modifié est une image (commence par 'img')
  if (name.startsWith('img')) {
    // Met à jour le state pour les champs d'image
    setArticle(prev => ({
      ...prev, // Garde toutes les propriétés précédentes
      picture: {
        ...prev.picture, // Garde toutes les images précédentes
        [name]: value // Met à jour uniquement l'image modifiée
      }
    }));
  } else {
    // Met à jour le state pour les champs non-image
    setArticle(prev => ({
      ...prev, // Garde toutes les propriétés précédentes
      [name]: value // Met à jour uniquement le champ modifié
    }));
  }
};
```

👤 : Nous allons maintenant créer la méthode "handleSubmit" qui permettra d'envoyer les données mises à jour vers la base de données.

```
// Fonction pour gérer la soumission du formulaire
const handleSubmit = async (e) => {
  // Empêche le comportement par défaut du formulaire
  e.preventDefault();
  try {
    // Envoie une requête PUT à l'API pour mettre à jour l'article
    const response = await axios.put(`http://localhost:8000/api/article/update/${id}`, article);

    // Si la mise à jour est réussie (statut 200)
    if (response.status === 200) {
      // Redirige vers la page de détail de l'article
      navigate(`/detail/${id}`);
    }
  } catch (error) {
    // Affiche l'erreur dans la console si la mise à jour échoue
    console.error('Erreur lors de la mise à jour:', error);
  }
};
```

👤 : Maintenant, nous pouvons mettre en place notre formulaire avec les données existantes ! Nous allons pré-remplir les champs du formulaire avec les données récupérées grâce à notre "useEffect"

```
<form onSubmit={handleSubmit}>
  <h1>Modifier l'article</h1>

  <div>
    <label>Nom:</label>
    <input
      type="text"
      name="name"
      value={article.name}
    />
  </div>
</form>
```

```

        onChange={handleChange}
      />
    </div>

    <div>
      <label>Contenu:</label>
      <textarea
        name="content"
        value={article.content}
        onChange={handleChange}
      />
    </div>

    <div>
      <label>Catégorie:</label>
      <input
        type="text"
        name="category"
        value={article.category}
        onChange={handleChange}
      />
    </div>

    <div>
      <label>Marque:</label>
      <input
        type="text"
        name="brand"
        value={article.brand}
        onChange={handleChange}
      />
    </div>

    <div>
      <label>Prix:</label>
      <input
        type="number"
        name="price"
        value={article.price}
        onChange={handleChange}
      />
    </div>

    {imgInputs.map((imgName, index) => (
      <div key={imgName}>
        <label>
          {index === 0 ?
            'Image principale (URL):'
            :
            `Image ${index} (URL):`}
        </label>
        <input
          type="text"
          name={imgName}
          value={article.picture[imgName] ? article.picture[imgName] : ''}
          onChange={handleChange}
        />
      </div>
    ))}

```

```

    </div>
  )))}

  <div>
    <label>Status:</label>
    <input
      type="checkbox"
      name="status"
      checked={article.status}
      onChange={(e) => setArticle(prev => ({
        ...prev,
        status: e.target.checked
      })))}
    />
  </div>

  <div>
    <label>Stock:</label>
    <input
      type="number"
      name="stock"
      value={article.stock}
      onChange={handleChange}
    />
  </div>

  <button type="submit">Mettre à jour</button>
</form>

```

👤 : Bravo si tu es arrivé jusqu'ici ! La modification d'article devrait être fonctionnelle maintenant. 🥳

PAGE REGISTER USER ⇒ 🤔

👤 : Créer une page "register", et dans cette page nous allons mettre en place le code pour s'inscrire en tant qu'utilisateur. Préparer la structure par défaut de votre composant.

👤 : Commençons par importer les éléments nécessaires

```

import React, { useState } from 'react'
import { Link } from 'react-router-dom'
import axios from 'axios'

```



Pensez à mettre la route pour ce composant.

👤 : Nous allons maintenant mettre en place un state "user" dans notre composant "Register"

```

const [user, setUser] = useState({
  isActive: tr

```

```
})
```

👤 : Ajoutons maintenant la méthode `"handleChange"` qui permettra de mettre à jour notre state `"user"`.

```
const handleChange = (event) => {  
  const { name, value } = event.target  
  setUser((user) => ({ ...user, [name]: value })))  
}
```

👤 : Créons ensuite la méthode `"handleSubmit"` qui enverra à l'API les données collectées par notre formulaire.

```
const handleSubmit = async (event) => {  
  // Empêche le comportement par défaut du formulaire  
  event.preventDefault()  
  try{  
    // Envoie une requête POST à l'API avec les données utilisateur  
    await axios.post('http://localhost:8000/api/user/signup', user)  
  }catch(e){  
    // En cas d'erreur, affiche l'erreur dans la console  
    console.log(e.message);  
  }  
}
```

👤 : Maintenant, nous pouvons mettre en place notre formulaire d'inscription avec les champs attendus dans la base de donnéesDD.

```
<form onSubmit={handleSubmit}>  
  <input  
    type='text'  
    onChange={handleChange}  
    placeholder='prenom'  
    name='prenom'  
  />  
  <input  
    type='text'  
    onChange={handleChange}  
    placeholder='URL picture'  
    name='avatar'  
  />  
  <input  
    type='text'  
    onChange={handleChange}  
    placeholder='email'  
    name='email'  
  />  
  <input  
    type='password'  
    onChange={handleChange}  
    name='password'  
  />  
  <button>Registration</button>  
</form>
```

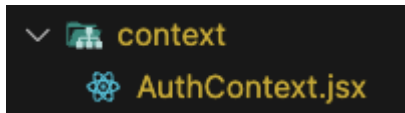
👤 : Pensez à ajouter la ligne ci-dessous pour rediriger l'internaute vers la page de connexion s'il est déjà inscrit.

```
<Link to='/sign'>Already registered ?</Link>
```

SIGN X CONTEXT ⇒ 🤔

👤 : Pour commencer, passons au Context. Si vous ne savez pas ce que c'est, récupérez le [PDF](#) qui contient les explications sur le Context et Redux.

Créez d'abord un dossier "context" dans le dossier "src", puis ajoutez-y un fichier "AuthContext" (attention à la majuscule).



👤 : Ajoutons les importations nécessaires :

```
import { createContext, useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";

// Pour communiquer avec notre API
import axios from "axios";
```

👤 : Nous devons maintenant créer le Context.

```
// Créez un context
export const AuthContext = createContext();
```

👤 : Mettez en place le composant "AuthProvider" et retournez le "AuthContext.Provider" pour fournir les données aux composants enfants.

```
export const AuthProvider = ({ children }) => {

  return (
    <AuthContext.Provider>
      {children}
    </AuthContext.Provider>
  );
};
```

👤 : Maintenant, ajoutons les states nécessaires pour gérer les étapes de la connexion, stocker les données de l'utilisateur connecté et gérer la redirection après connexion.

```
// Etat pour suivre l'authentification
const [isLoading, setIsLoading] = useState(false);

// Etat pour stocker les infos de l'user connecté
const [auth, setAuth] = useState(null);
```

```
// Navigate
const navigate = useNavigate()
```

👤 : Mettons en place la fonction "login" qui se chargera de récupérer les données provenant de votre futur formulaire de connexion.

```
const login = async (dataForm) => {
  // Active l'indicateur de chargement
  setIsLoading(true);
  try {
    // Envoi d'une requête POST à l'API avec les données du formulaire
    const { data, status } = await axios.post(`http://localhost:8000/api/user/sign`, dataForm);

    // Si la requête est réussie (status 200)
    if (status === 200) {
      // Sauvegarde les données de l'utilisateur dans le localStorage pour les conserver
      localStorage.setItem("auth", JSON.stringify(data));

      // Met à jour le state avec les données de l'utilisateur connecté
      setAuth(data);

      // Redirige l'utilisateur vers la page d'accueil
      navigate('/')

      // Désactive l'indicateur de chargement car l'authentification est terminée
      setIsLoading(false);
    }
  } catch (e) {
    // En cas d'erreur, affiche le message dans la console
    console.log(e.message);
    // Désactive l'indicateur de chargement car le processus est terminé
    setIsLoading(false);
  }
};
```

👤 : Ajoutons au "AuthContext.Provider" les données à fournir (nous aurons besoin de récupérer la fonction "login" et les states "auth" et "isLoading" dans les composants enfants).

```
<AuthContext.Provider value={{ login, auth, isLoading }}>
  {children}
</AuthContext.Provider>
```

👤 : Mettons en place le Context au plus haut niveau de notre application, dans le fichier "index.js" ajouter ces ligne dans le fichier.

```
// CONTEXT
import { AuthProvider } from './context/AuthContext';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```


👤 : Maintenant que notre **Context** fournit les données aux **composants** enfants, créons le fichier "**sign.js**". N'oubliez pas la structure de base et sa route "**signup**".

👤 : Importons les éléments nécessaires dans le composant "**signup**".

```
// useContext (pour accéder au contexte d'authentification)
import React, { useState, useContext } from 'react'

import { Link } from 'react-router-dom'
// On importe le contexte d'authentification AuthContext
import { AuthContext } from '../context/AuthContext'
```

👤 : Mettons en place le state "**user**" et récupérons la fonction "**login**".

```
const [user, setUser] = useState({})
const { login } = useContext(AuthContext)
```

👤 : Ajoutons la fonction "**handleChange**".

```
const handleChange = event => {
  const { name, value } = event.target
  setUser(prevUser => ({...prevUser, [name]: value}))
}
```

👤 : Puis ajoutons la fonction "**handleSubmit**". C'est ici qu'on appellera la fonction "**login**" créée dans le **Context**, en lui fournissant les données collectées par le formulaire.

```
const handleSubmit = event => {
  event.preventDefault()
  // CALL CONTEXT
  login(user)
}
```

👤 : Le formulaire à mettre en place.


```
<form onSubmit={handleSubmit} >
  <label htmlFor="email">Email : </label>
  <input
    type='email'
    placeholder='email'
    name='email'
    id='email'
    onChange={handleChange}
  />
  <label htmlFor="password">Password : </label>
  <input
    type="password"
    name="password"
    id="password"
    onChange={handleChange}
  />
  <button>Login</button>
</form>
```

 : Enfin, n'oubliez pas d'ajouter le lien pour les utilisateurs non enregistrés.

```
<Link to='/register'> You are not registered ? </Link>
```

REDUX ⇒ 🤒

19 ⇒ 🧑 : Mettons en place Redux dans notre projet. Pour cela, commençons par installer les dépendances dans votre application :

```
npm i react-redux @reduxjs/toolkit
```

 : Installez l'extension Redux DevTools pour analyser le store de votre application, Pour chrome.

Redux DevTools - Chrome Web Store

Redux DevTools for debugging application's state changes.

 <https://chromewebstore.google.com/detail/redux-devtools/lmhkpmbekcpmknkloeibfkpmmfiblj?hl=en>

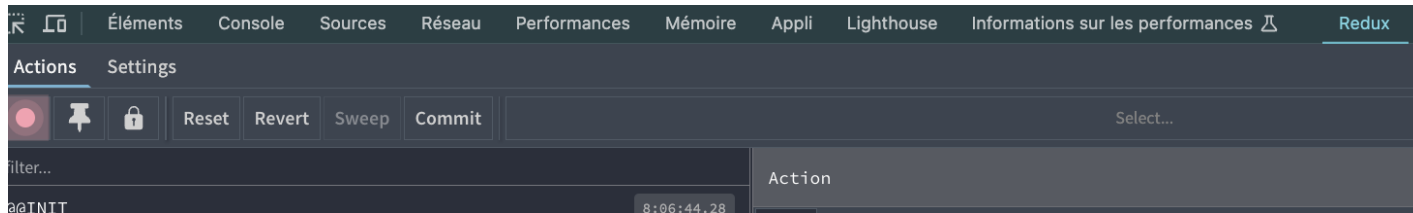
 : Firefox ⇒

Redux DevTools - Chrome Web Store

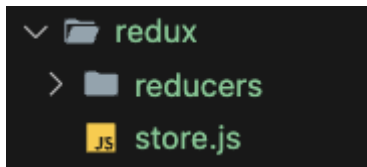
Redux DevTools for debugging application's state changes.

 <https://chromewebstore.google.com/detail/redux-devtools/lmhkpmbekcpmknklieibfkpmmfiblj?hl=fr>

 : Redémarrer votre navigateur vous devrez avoir ça dans les onglet de l'inspecteur :



👤 : Créez un dossier "Redux" qui doit contenir un dossier "reducers" et un fichier "store.js"



👤 : Dans le fichier "store.js" nous allons mettre en place la configuration.

```
// Import de la fonction configureStore depuis Redux Toolkit pour créer le store
import { configureStore } from '@reduxjs/toolkit'

// Configuration et export du store Redux
// configureStore crée automatiquement le store avec les bonnes configurations
export default configureStore({
  // Définition des reducers de l'application
  // Chaque clé correspond à une partie du state global
  reducer: {

  }

})
```

👤 : Nous avons besoin de relier le store à notre application. Nous allons le faire au plus haut niveau de notre application dans le fichier "index.js", ajoutons les importations "store" et "Provider".

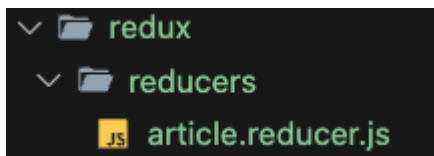
```
// Import du store Redux qu'on a créé pour gérer l'état global de l'application
import store from './redux/store'

// Import du Provider de React-Redux qui va envelopper notre app
// pour donner accès au store à tous les composants
import { Provider } from 'react-redux'
```

👤 : Enveloppons les composants de notre application en leur fournissant le store.

```
<React.StrictMode>
  <BrowserRouter>
    <Provider store={store}>
      <AuthProvider>
        <App />
      </AuthProvider>
    </Provider>
  </BrowserRouter>
</React.StrictMode>
```

👤 : Maintenant que notre application peut accéder au store, nous pouvons commencer notre reducer article 🤖, du coup dans le dossier "reducer" créons un fichier "article.reducer.js"



👤 : On **importe** le package '@reduxjs/toolkit'.

```
// Import de createSlice depuis Redux Toolkit - permet de créer un reducer avec moins de code
import { createSlice } from '@reduxjs/toolkit'
```

👤 : Définissons les données initiales de notre **store**.

```
// Définition de l'état initial du reducer
// data: tableau vide qui contiendra nos articles
// loading: pour gérer l'état de chargement
// error: pour gérer les erreurs
const initialState = {
  data: [],
  loading: null,
  error: false
}
```

👤 : Ajouter ceci ⇒

```
// Création du slice pour les articles
export const Article = createSlice({
  name: "Article", // Nom du slice
  initialState,    // État initial défini plus haut
  reducers: {

  }
})
```

👤 : Mettons en place votre première action. Les actions sont très importantes car elles permettent de communiquer avec notre reducer.



Le **store** représente une copie de **initialState**, on y trouvera toutes les propriétés, data , loading, error.

Les actions (FETCH_ARTICLE_START) sont simplement des messages qui indiquent au store quels changements faire. C'est comme donner des instructions précises pour mettre à jour les données de l'application.

- Chaque action a un nom unique qui dit ce qu'elle fait
- Elle peut aussi contenir des données si nécessaire

C'est la seule façon de modifier les données dans Redux.

```
// Action déclenchée au début du chargement des articles
FETCH_ARTICLE_START: (store) => {
  store.loading = true // On met loading à true pendant le chargement
},
```



👤 : Le **payload** est un paramètre qui contient les données que vous souhaitez transmettre à votre **action**. C'est la charge utile qui accompagne l'**action** pour modifier le **state**.

Par exemple, dans une action de succès comme **FETCH_ARTICLE_SUCCESS**, le **payload** contiendrait les articles récupérés depuis l'**API**.

L'action utilise ensuite ces données pour mettre à jour le state avec les nouveaux articles

Le **payload** peut contenir n'importe quel type de données (objets, tableaux, chaînes de caractères, etc.) nécessaires pour effectuer la modification du state souhaitée.

```
// Action déclenchée quand les articles sont bien récupérés
FETCH_ARTICLE_SUCCES: (store, actions) => {
  store.loading = false      // Chargement terminé
  store.data = actions.payload // On stocke les articles reçus dans data
}
```

👤 : N'oubliez pas d'exporter vos actions et votre reducer afin de pouvoir les utiliser dans vos différents composants.

```
// Export des actions pour pouvoir les utiliser dans nos composants
export const { FETCH_ARTICLE_START, FETCH_ARTICLE_SUCCES } = Article.actions
// Export du reducer pour l'ajouter au store
export default Article.reducer
```

👤 : Ajoutez le reducer "**Article**" dans le configureStore défini dans le fichier "**store.js**".

```
// Import de la fonction configureStore depuis Redux Toolkit pour créer le store
import { configureStore } from '@reduxjs/toolkit'

// Import des reducers - ici on importe le reducer Article depuis son fichier
import Article from './reducers/article.reducer'

// Configuration et export du store Redux
// configureStore crée automatiquement le store avec les bonnes configurations
export default configureStore({
  // Définition des reducers de l'application
  // Chaque clé correspond à une partie du state global
  reducer: {
    article: Article // Le reducer Article gérera la partie "article" du state
  }
})
```

👤 : Importons les éléments nécessaires dans la page '**home.js**'

```
import { useDispatch, useSelector } from 'react-redux'
// Importation des actions de Redux
import * as ACTIONS from '../redux/reducers/article.reducer'
```

👤 : Mettons en place la constante dispatch dans notre composant.

Grâce à ce hook, nous pourrons appeler les actions de notre reducer.

```
// utilisation du hook useDispatch pour gérer les actions de Redux
const dispatch = useDispatch()
```

👤 : Appelons l'action `FETCH_ARTICLE_START` grâce à notre dispatch.

```
const fetchArticles = async () => {
  // Dispatch de l'action de démarrage
  dispatch(ACTIONS.FETCH_ARTICLE_START())

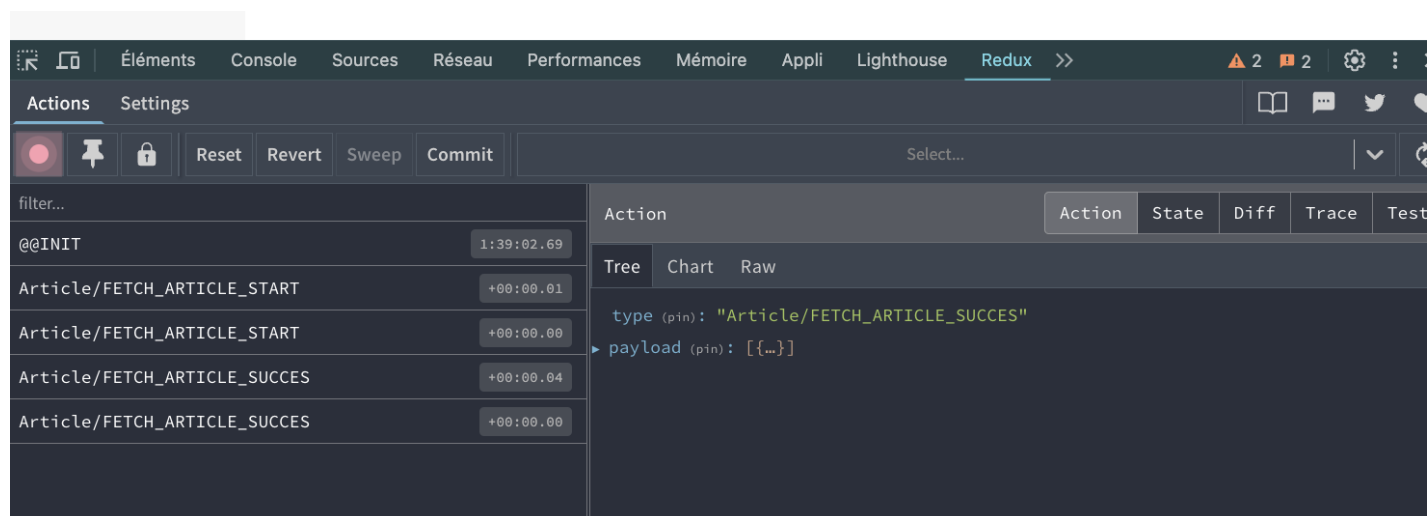
  try{
```

👤 : Déconstruisons l'appel à notre API en récupérant `data` et `status`, puis envoyons nos données à notre `store` en appelant l'action `FETCH_ARTICLE_SUCCESS` avec les données récupérées de notre API.

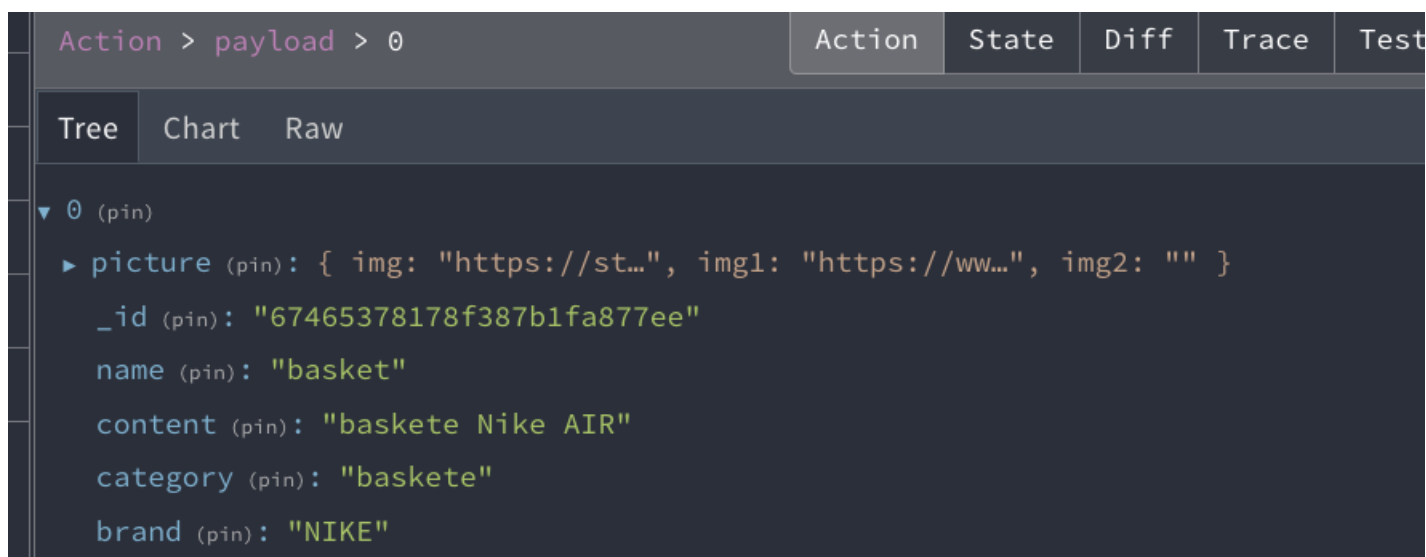
```
// HOOK useEffect POUR GÉRER LES ARTICLES
useEffect(() => {
  const fetchArticles = async () => {
    // Dispatch de l'action de démarrage
    dispatch(ACTIONS.FETCH_ARTICLE_START())

    try{
      // Récupère les articles
      const { data, status } = await axios.get(`http://localhost:8000/api/article/all`)
      dispatch(ACTIONS.FETCH_ARTICLE_SUCCES(data))
    }catch(e){
      console.log(e.message);
    }
  }
  fetchArticles()
}, [])
```

👤 : Observez votre store à partir de l'extension installée - vous devriez avoir le résultat ci-dessus.



👤 : Cliquez sur "`payload`" pour visualiser le contenu de votre store.



👤 : Le `useSelector` permet de récupérer les données dans notre `store`, vous pouvez cliquer sur "`state`" à partir de votre extension pour voir le chemin à parcourir.

```
// utilisation du hook useDispatch pour gérer les actions de Redux
const dispatch = useDispatch()

const store = useSelector(state => state.article.data)
```

👤 : Maintenant nous pouvons parcourir notre store avec la méthode `map` pour afficher les données qu'il contient. (Le state "`article`" n'est donc plus nécessaire)

```
{store && store.map(article => (
  <div key={article._id}>
    <p>{article.name}</p>
    <img src={article.picture.img} alt='' width={200} />
  </div>
))}
```

FEATURE USER

👤 : Créer une page "`Dashboard.js`" cette interface sera accessible que pour l'administrateur de votre application.

1 ⇒ Mettez en place votre CRUD (create, read, update, delete) et n'oubliez pas que les données affichées doivent provenir du `store` (Inspiré vous de votre crud article, si vous aviez survécu jusqu'ici ça devrait aller 😊). (**Ne perdez pas de temps avec le CSS**) faites-le une fois qu'il y aura toutes les Fonctionnalités.

Par exemple, un Dashboard peut ressembler à ceci :

CRUD VOITURE

vehicule	Agence	titre	marque	modele	description	photo	prix	actions
1508	Paris	BMW Série3	BMW	Série3	Diesel, boîte auto, 5 portes, GPS, AutoRadio, Forfait 1000 km (0,5/km supplémentaire)		200 €	  
1511	Paris	Audi A1	Audi	A1	Diesel, 5 portes, GPS, AutoRadio, Forfait 1000 km (0,55/km supplémentaire)		150 €	  
1532	Paris	Peugeot 208	Peugeot	208	Diesel, 5 portes, GPS, AutoRadio, Forfait 1000 km (0,5/km supplémentaire)		120 €	  

Titre

Titre de l'annonce

Marque

Marque

Modele

Modele

Prix

Prix journalier

Photo

PHOTO 1



Description

Description de votre vehicule

Enregistrer

CRUD AGENCE

BackOffice / Gestion des Agences

Agence	titre	adresse	ville	cp	description	photo	actions
33	Agence de Paris	300 boulevard de vaugirard	Paris	75015	Notre Agence de Paris 300 boulevard de vaugirard, 75015, Paris Ouvert 7j/7 de 09h à 13h et de 14h à 20h 01 49 78 21 33 contact@veville.com		  
68	Agence Lyonnaise	15 rue sainte-catherine	Lyon	69003	Agence de Lyon ...		  
77	Agence de Bordeaux	10 Place de l'hôtel de ville	Bordeaux	33000	Notre Agence de Bordeaux vous ouvre ces portes ...		  

Titre

Titre de l'agence

Description

Description de votre agence

Photo

PHOTO 1



Adresse

Adresse

Ville

Ville

Code Postal

Code Postal

Enregistrer

CRUD MEMBRES

id membre	pseudo	nom	prenom	email	civilite	statut	date_enregistrement	actions
1	admin	Marie	Thoyer	marie.thoyer@gmail.com	Femme	admin	06/06/2016 14:45	  
2	joker	Cottet	Julien	juju70@gmail.com	Homme	membre	06/06/2016 20:30	  
3	camelus	Miller	Guillaume	guillaume-miller@gmail.com	Homme	membre	07/06/2016 09:30	  

Pseudo

 pseudo

Email

 votre email

Mot de passe

 mot de passe

Civilité

Homme 

Nom

 votre nom

Statut

Admin 

Prénom

 votre prénom

Enregistrer

BON COURAGE 🍷🍷